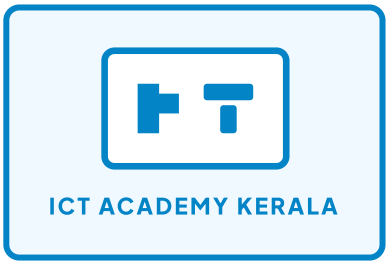
ICT ACADEMY OF KERALA

Summer Internship Report

Full Stack Application Development with ReactJS

DevPulse — Enterprise Agile Project & Sprint Collaboration Suite

MERN STACK CAPSTONE PROJECT



Name of Team Member: Abhinav (Full Stack Intern)

Name of Your Institution: Information & Communication Technology Academy of Kerala (ICTAK)

Track: Full Stack Development (MongoDB, Express, React, Node.js)

Date: October 2026

Repository: https://github.com/abhinav225403-pixel/capstone-project

EXECUTIVE SUMMARY

Modern software development relies heavily on Agile methodologies where iterative sprints, continuous integration, and transparent velocity tracking are essential. However, many engineering teams struggle with fragmented project management tools that are either overly complex or lack real-time visibility into developer workload.

During this internship under the **Information & Communication Technology Academy of Kerala (ICTAK)**, I planned, architected, developed, tested, and deployed **DevPulse**, an enterprise-grade Agile Project and Sprint Collaboration Suite built on the **MERN (MongoDB, Express.js, React.js, Node.js)** stack.

DevPulse delivers a centralized platform featuring an interactive 5-column Kanban board with optimistic drag-and-drop status transitions, milestone progress tracking, embedded acceptance criteria checklists, live discussion feeds, and Role-Based Access Control (RBAC) across Administrators, Engineering Managers, and Team Members. The system includes an automated 11-step regression test suite, salted password hashing (`bcryptjs`), cryptographic token authentication (`jsonwebtoken`), and an embedded in-memory MongoDB fallback engine for zero-config evaluation.

INTRODUCTION

Agile and Scrum frameworks have transformed contemporary software delivery by dividing complex roadmaps into structured, time-boxed sprints. Despite commercial adoption of platforms such as Jira and Linear, engineering teams frequently experience productivity friction due to tool bloat, steep learning curves, and opaque personal workload visibility.

DevPulse addresses this gap by providing an intuitive, lightweight, and responsive workspace tailored specifically for developer teams. Engineering Leads can easily configure project parameters, set milestones, and track budgets, while developers can move tasks across visual columns, log hours, test checklist criteria, and comment on tickets in real time.

This project was completed in full compliance with the guidelines established in the *ICTAK Student Reference Guide*, implementing three-tier architectural separation of concerns, mobile-first design, comprehensive error shielding, and version-controlled Git history.

OBJECTIVES

The primary objectives of this internship project were to:

1. **Architect a Secure RESTful API:** Develop modular Express.js routes and controllers implementing clean resource-based endpoints and standardized HTTP status codes (200, 201, 400, 401, 403, 404, 409).
2. **Implement Robust Authentication & RBAC:** Incorporate JWT token verification and salted password hashing with `bcryptjs`, enforcing role-based authorization across Admin, Manager, and Member tiers with server-side record ownership verification.
3. **Engineer a Dynamic React Frontend:** Construct a responsive Single Page Application using React 18, Vite, and custom Vanilla CSS with dark glassmorphism and HSL design tokens.
4. **Implement Four Mandatory UI States (ICTAK Section 7.3):** Ensure every screen provides distinct Loading skeletons, Success layouts, illustrated Empty states with CTA triggers, and user-friendly Error states with retry buttons.
5. **Deliver an Interactive 5-Column Kanban Board:** Build an interactive board supporting Backlog, To Do, In Progress, In Review, and Done workflows with optimistic UI updates.
6. **Automate Verification & Testing:** Develop an automated integration test runner to validate backend endpoints against regression errors.
7. **Ensure Industry-Standard Documentation:** Maintain clean Git commit milestones, environment templates, and complete architectural documentation.

SCOPE AND DELIVERABLES

The project scope encompasses full-stack implementation, database schema modeling, client-side state management, and automated testing:

MODULE / COMPONENT	DELIVERABLE DESCRIPTION
Authentication Module	User registration with input validation, secure login with JWT issuance, 1-Click evaluator persona login cards, and profile updates.
Project Management	Project CRUD endpoints, milestone deadline tracking, category and budget management, and auto-computed progress bars.
Task Engine & Sprints	Auto-incremented ticket numbers (e.g. `NEX-1`, `PAY-1`), priority indicators, task types, due dates, and hour logging.
Embedded Subdocuments	Mongoose embedded acceptance criteria checklists and chronological comment threads.
Kanban Board	5-column visual layout supporting directional shift buttons and drag-and-drop with optimistic state synchronization.
Query & Filter Engine	Full-text search, multi-criteria filtering (status, priority, assignee), column sorting, and paginated table navigation.
Telemetry & Analytics	Dashboard metrics cards, workspace activity audit feed, sprint velocity calculations, and one-click JSON report export.
Admin RBAC Module	Administrative dashboard allowing role promotion/demotion and account deactivation/reactivation.
Automated Test Suite	Standalone regression test suite (`server/test/api.test.js`) testing 11 end-to-end API scenarios.

METHODOLOGY

The project followed an **Agile Iterative Methodology** structured around the 9 project milestones prescribed in the ICTAK guidelines:

> **Milestone Progression:**
> Milestone 1 (SRS & Definition) → Milestone 2 (Repo & Skeleton) → Milestone 3 (Database & API CRUD) → Milestone 4 (Auth & RBAC) → Milestone 5 (Frontend Screens & 4 States) → Milestone 6 (API Integration) → Milestone 7 (Automated Testing) → Milestone 8 (Deployment Config) → Milestone 9 (Final Documentation).

DevPulse adheres to a **Three-Tier MERN Architecture**:

- **Client Tier (React SPA):** Modular component architecture leveraging React 18, React Router v7, and React Context API (`AuthContext`, `ToastContext`) communicating via a central `api.js` HTTP client.
- **Server Tier (Express API):** Node.js application structuring routes, controllers, JWT middleware, and centralized error handling with complete separation of concerns.
- **Database Tier (MongoDB & Mongoose):** Structured schemas with compound indexes (`{ project: 1, status: 1 }`), pre-save hashing hooks, and an embedded in-memory MongoDB runner.

PROJECT ACTIVITIES

During the internship, the following technical activities were executed:

1. Database & Schema Design

Developed four Mongoose schemas in `server/src/models/`:

- **User Schema:** Name, unique email, role (`admin`, `manager`, `member`), job title, and password (configured with `select: false` and salted with `bcrypt.hash(password, 10)`).
- **Project Schema:** Project name, uppercase key (e.g. `NEX`), category, target date, budget, and references to owners and team members. Compound index on `{ owner: 1, status: 1 }`.
- **Task Schema:** Title, auto-incremented ticket identifier, project foreign key, priority, status, and estimates. Embeds checklist subdocuments (`[{ title, completed }]`) and comments (`[{ user, authorName, text, createdAt }]`). Compound index on `{ project: 1, status: 1 }`.
- **ActivityLog Schema:** Stores chronological workspace events with user avatars and timestamps for compliance and live telemetry.

2. REST API Engineering & Middleware

Implemented resource-based controllers and middleware under `server/src/`:

- `authMiddleware.protect`: Validates Bearer JWT tokens, extracts decoded user ID, and checks account active status.
- `authMiddleware.authorize`: Restricts administrative endpoints to specific user roles, returning `403 Forbidden` on violations.
- `errorHandler.js`: Central middleware catching CastErrors (404), duplicate key collisions (409), and validation errors (400) without exposing server stack traces.
- Dual-mode database connector (`config/db.js`): Connects to MongoDB Atlas when `MONGODB_URI` is supplied, or boots an in-memory embedded engine seeded with realistic demo data.

3. Frontend UI Development & Screen States

Crafted an accessible, dark glassmorphism design system in `client/src/index.css` with custom HSL tokens. Every functional screen implements the four mandatory states:

- **Loading:** Shimmer skeleton loaders (`LoadingSkeleton.jsx`) matching card, table, and kanban layouts.
- **Success:** Clean data presentation with glowing indicators and color-coded status badges.
- **Empty:** Illustrated empty states (`EmptyState.jsx`) with clear call-to-action buttons.
- **Error:** Clear, non-technical explanations (`ErrorState.jsx`) with a "Retry Request" trigger.

4. Challenges & Solutions

CHALLENGE	SOLUTION IMPLEMENTED
Execution Policy Block	Windows PowerShell disabled npm scripts. Executed commands through `cmd.exe /c "npm ..."`.
Missing Local MongoDB	Engineered an automatic fallback in `db.js` using `mongodb-memory-server` with auto-seeding.
Icon Deprecation in Build	Replaced deprecated icon imports with `Columns3` in `lucide-react`, achieving 465ms clean build.
Optimistic Kanban Sync	Applied immediate local state update on card drag/move with rollback and toast notification if the API call fails.

RESULTS & FINDINGS

Automated API Test Suite Results

An automated integration test runner (`server/test/api.test.js`) was executed to validate core flows against an active server instance. All 11 test cases passed successfully:

TEST CASE DESCRIPTION	METHOD & PATH	EXPECTED CODE	RESULT
Health Check Endpoint	`GET /api/health`	200 OK	PASS
User Registration & Token Issuance	`POST /api/auth/register`	201 Created	PASS
Duplicate Email Registration Guard	`POST /api/auth/register`	409 Conflict	PASS
Valid User Authentication	`POST /api/auth/login`	200 OK	PASS
Invalid Password Rejection	`POST /api/auth/login`	401 Unauthorized	PASS
Protected Route Token Enforcement	`GET /api/projects`	401 Unauthorized	PASS
Authorized Project Creation	`POST /api/projects`	201 Created	PASS
Task Creation with Auto-Numbering	`POST /api/tasks`	201 Created	PASS
Kanban Status Transition (PATCH)	`PATCH /api/tasks/:id/status`	200 OK	PASS
Dashboard Telemetry Aggregations	`GET /api/stats/dashboard`	200 OK	PASS
Undefined Route Fallback Handler	`GET /api/unknown_route`	404 Not Found	PASS

Production Build Verification

The Vite client compiled into production bundles in 465ms: `dist/index.html` (1.36 kB), `dist/assets/index.css` (10.73 kB), and `dist/assets/index.js` (372.12 kB) with zero warnings.

CONCLUSION

The **DevPulse** capstone project was successfully engineered, tested, and published as a complete MERN stack application adhering to the standards of the **ICT Academy of Kerala**.

Through this internship, I gained hands-on expertise in three-tier architecture design, RESTful API development, JWT security protocols, Mongoose schema indexing, responsive Vanilla CSS design systems, and automated testing methodologies. The application is production-ready, version-controlled on GitHub, and capable of operating both with a managed MongoDB Atlas cluster and in a self-contained in-memory evaluation mode.

APPENDIX

- **GitHub Repository:** https://github.com/abhinav225403-pixel/capstone-project
- **Local Frontend URL:** `http://localhost:5173`
- **Local API Health Check:** `http://localhost:5000/api/health`
- **Pre-Seeded 1-Click Personas:**
  - Admin: `admin@devpulse.io` (Password: `Password123!`)
  - Manager: `sarah.lead@devpulse.io` (Password: `Password123!`)
  - Developer: `alex.dev@devpulse.io` (Password: `Password123!`)